

PROJECT UTS – STRUKTUR DATA

Sistem Undo dan Redo Menggunakan Stack (LIFO)

1. Identitas Kelompok

- Nama: I Gede Japa Tuan
 - NIM: 2501010221
 - GitHub: Japa-67
 - Nama: Ach. Rafli Alfiansyah
 - NIM: 2501010048
 - GitHub: raflialfiansyah70-fly
-

2. Rumusan Masalah

1. Bagaimana konsep struktur data stack dapat diterapkan dalam sistem undo dan redo pada aplikasi?
2. Bagaimana implementasi stack menggunakan array dalam membangun sistem undo dan redo?
3. Bagaimana sistem yang dibuat mampu menyelesaikan permasalahan pengelolaan riwayat aksi pengguna secara efisien dan terstruktur?

Pembahasan dan Solusi

Sistem menggunakan dua stack (undo dan redo) dengan prinsip LIFO, sehingga aksi terakhir dapat dibatalkan dan dikembalikan kembali. Dalam pengembangan sistem berbasis interaksi pengguna, seperti editor teks atau aplikasi desain, sering kali dibutuhkan fitur untuk membatalkan aksi terakhir (*undo*) maupun mengembalikannya kembali (*redo*). Permasalahan utama yang muncul adalah bagaimana mengelola riwayat aksi tersebut secara terstruktur, efisien, dan sesuai urutan waktu.

Untuk menjawab permasalahan tersebut, digunakan struktur data **stack** dengan prinsip **LIFO (Last In First Out)**. Dalam sistem ini, digunakan dua buah stack, yaitu *undo stack* dan *redo stack*.

- Setiap aksi baru yang dilakukan pengguna akan disimpan ke dalam *undo stack*.

- Ketika pengguna melakukan undo, data terakhir dari *undo stack* akan dipindahkan ke *redo stack*.
- Ketika pengguna melakukan redo, data dari *redo stack* akan dikembalikan ke *undo stack*.

Pendekatan ini memungkinkan sistem untuk mengelola riwayat aksi secara dinamis dan memastikan bahwa operasi undo dan redo berjalan secara konsisten.

3. Landasan Teori

Struktur data merupakan cara untuk mengorganisir, menyimpan, dan mengelola data agar dapat digunakan secara efisien dalam suatu sistem. Pemilihan struktur data yang tepat akan sangat mempengaruhi kinerja dan efisiensi program yang dikembangkan.

Salah satu jenis struktur data yang sering digunakan adalah **stack**. Stack adalah struktur data linear yang bekerja berdasarkan prinsip **LIFO (Last In First Out)**, di mana elemen terakhir yang dimasukkan akan menjadi elemen pertama yang dikeluarkan. Operasi utama dalam stack meliputi *push* (menambahkan data), *pop* (menghapus data), dan *peek* (melihat elemen teratas).

Konsep LIFO sangat relevan dalam berbagai aplikasi nyata, salah satunya adalah fitur undo dan redo. Dalam sistem ini, setiap aksi pengguna dicatat dan dapat dibatalkan atau dikembalikan sesuai urutan terakhirnya. Hal ini menjadikan stack sebagai struktur data yang tepat untuk mengimplementasikan fitur tersebut.

Dalam implementasinya, stack dapat dibangun menggunakan **array** atau **linked list**. Pada penelitian ini, digunakan array karena lebih sederhana dan mudah diimplementasikan dalam bahasa pemrograman Python. Array memungkinkan penyimpanan data secara berurutan dan mendukung operasi push dan pop dengan efisien.

Daftar Pustaka

- Cormen, T. H. (2009). Introduction to Algorithms. MIT Press
 - Weiss, M. A. (2014). Data Structures and Algorithm Analysis. Pearson
 - Sedgewick, R. (2011). Algorithms. Addison-Wesley
-

4. Desain Sistem

Studi Kasus Nyata

Sistem yang dirancang dalam proyek ini adalah simulasi fitur **undo dan redo pada aplikasi editor teks**. Dalam aplikasi tersebut, pengguna dapat melakukan berbagai aksi seperti mengetik, menghapus, atau mengubah teks, dan sistem harus mampu menyimpan serta mengelola riwayat aksi tersebut.

Alur Sistem (Input – Proses – Output)

- **Input:** Aksi yang dilakukan oleh pengguna (misalnya mengetik teks)
- **Proses:**
 - Aksi disimpan ke dalam *undo stack*
 - Jika undo dilakukan, data dipindahkan ke *redo stack*
 - Jika redo dilakukan, data dikembalikan ke *undo stack*
- **Output:** Riwayat aksi pengguna ditampilkan

Flowchart

Start



Input Aksi



Push ke Undo Stack



Pilih Aksi?



Undo



Redo



Tampil



Pindah



Balik



Tampilkan

Stack

Stack



End

Pseudocode

push(data):

tambah ke undo

kosongkan redo

undo():

pindah dari undo ke redo

redo():

pindah dari redo ke undo

5. Implementasi Program (Python)

```
undo_stack = []
redo_stack = []

def push(action):
    undo_stack.append(action)
    redo_stack.clear()

def undo():
    if undo_stack:
        action = undo_stack.pop()
        redo_stack.append(action)
        return action
    return "Tidak ada aksi"

def redo():
    if redo_stack:
        action = redo_stack.pop()
        undo_stack.append(action)
        return action
    return "Tidak ada aksi"

def show():
    print("Undo:", undo_stack)
    print("Redo:", redo_stack)

while True:
    print("\n1. Tambah Aksi")
    print("2. Undo")
    print("3. Redo")
    print("4. Lihat Stack")
    print("5. Keluar")

    pilih = input("Pilih: ")

    if pilih == "1":
        aksi = input("Aksi: ")
        push(aksi)
    elif pilih == "2":
        print("Undo:", undo())
    elif pilih == "3":
        print("Redo:", redo())
    elif pilih == "4":
        show()
    elif pilih == "5":
        break
```

Program di atas menggunakan dua list Python sebagai representasi stack. Operasi *append()* digunakan sebagai push, sedangkan *pop()* digunakan untuk mengambil elemen terakhir. Sistem juga mengatur agar *redo stack* dikosongkan setiap kali ada aksi baru, sehingga menjaga konsistensi alur undo dan redo.

6. Operasi Stack

Operasi yang digunakan dalam sistem ini meliputi:

- **Push:** Menambahkan aksi ke dalam undo stack
- **Pop (Undo):** Menghapus aksi terakhir dari undo stack dan memindahkannya ke redo stack
- **Pop (Redo):** Mengambil aksi dari redo stack dan mengembalikannya ke undo stack
- **Display:** Menampilkan isi kedua stack

7. Kesimpulan

Berdasarkan hasil perancangan dan implementasi sistem undo dan redo menggunakan struktur data stack, dapat disimpulkan bahwa seluruh rumusan masalah yang diajukan telah berhasil diselesaikan. Sistem yang dikembangkan mampu menunjukkan bagaimana konsep stack diterapkan dalam pengelolaan riwayat aksi pengguna secara terstruktur.

Sistem juga telah berjalan sesuai dengan teori struktur data, khususnya prinsip LIFO (Last In First Out), di mana aksi terakhir yang dimasukkan akan menjadi yang pertama diproses dalam operasi undo. Penggunaan dua stack memungkinkan proses undo dan redo berjalan secara sistematis dan konsisten.

Manfaat dari penerapan stack dalam sistem ini adalah memberikan kemudahan bagi pengguna dalam mengelola perubahan yang dilakukan, baik untuk membatalkan maupun mengembalikan aksi. Sistem ini memiliki relevansi tinggi dalam berbagai aplikasi nyata seperti editor teks, aplikasi desain grafis, dan sistem berbasis riwayat aktivitas lainnya.

8. Link GitHub

https://github.com/Japa-67/UTS_StrukturData